

Tehtävä 6: Spring Boot Mikropalvelut

Spring Cloud -mikropalveluarkkitehtuurin toteutus

a) Tutoriaalin läpikäynti

Monoliittisovelluksen ongelmat

Mikropalvelut pyrkivät ratkaisemaan seuraavia monoliittiarkkitehtuurin ongelmia:

- Skaalautuvuus: koko sovellus täytyy skaalata, vaikka vain yksi osa olisi kuormittunut
- Pitkät julkaisusykli: pienikin muutos vaatii koko sovelluksen uudelleenjulkaisun
- Teknologia riippuvuus: kaikki osat käyttävät samaa teknologiapinoa
- Vikaantumisen leviäminen: yksi virhe voi kaataa koko järjestelmän
- Tiimien riippuvuudet: useat tiimit työskentelevät samaan koodipohjaan

Mikropalveluarkkitehtuurin haasteet

- Palveluiden löytäminen vaatii erillisen naming servicen (Eureka)
- Hajautettu konfiguraatio on monimutkaisempaa hallita
- Verkkolatenssi ja virheen käsittely palveluiden välillä
- Monitorointi ja lokitus hajautuvat usealle palvelulle
- Monimutkaisempi infrastruktuuri kuin monoliitissa

Sovelluksen toiminta ja arkkitehtuuri

Sovellus muuntaa valuuttoja. Esimerkiksi pyyntö /currency-converter/from/USD/to/INR/quantity/10 palauttaa tuloksen 650 INR. Arkkitehtuuri koostuu seuraavista komponenteista:

Palvelu	Portti	Tehtävä
Eureka Naming Service	8761	Palvelurekisteri – kaikki muut palvelut rekisteröityvät tähän
Forex-service (x2)	8000, 8001	Tarjoaa valuuttakursseja H2-tietokannasta
Currency-conversion-service	8100	Vastaanottaa muuntopyynnön ja laskee tuloksen
Zuul API Gateway	8762	Yksi sisäänkäynti kaikkiin palveluihin
Image-service	8200	Tarjoaa kuvia valuutoille

Feign Client, Load Balancer ja Naming Service

Eureka Naming Service on palvelurekisteri, johon kaikki palvelut rekisteröityvät käynnistyessään. Näin palvelut löytävät toisensa nimen perusteella ilman kovakoodattuja osoitteita.

Feign Client on deklarativinen HTTP-asiakaskirjasto. Sen sijaan että kirjoitettaisiin manuaalinen HTTP-kutsukoodi, riittää rajapinta `@FeignClient`-annotaatiolla. Feign kysyy Euralta palvelun sijainnin automaattisesti.

Ribbon Load Balancer jakaa pyynnöt forex-servicen kahden instanssin (portit 8000 ja 8001) välille round-robin-periaatteella. Tämä todentuu vastauksen port-kentästä, joka vaihtelee pyynnöstä toiseen.

b) Zuul-gateway ja Image-service

Gatewayn merkitys

Zuul API Gateway toimii yhtenä sisäänkäyntinä kaikkiin mikropalveluihin portissa 8762. Ilman gatewaya asiakkaan pitäisi tietää jokaisen palvelun portti erikseen. Gateway mahdollistaa myös keskitetyn autentikoinnin, lokituksen ja rate limitingin.

Gatewayn kautta pyynnöt reitittyvät palvelun nimen perusteella:

- `http://localhost:8762/currency-conversion-service/currency-converter/...` → portti 8100
- `http://localhost:8762/image-service/image-service/USD` → portti 8200
- `http://localhost:8762/forex-service/currency-exchange/...` → portti 8000 tai 8001

Puuttuvat mikropalveluarkkitehtuurin ominaisuudet

Toteutetusta arkkitehtuurista puuttuu vielä seuraavia ominaisuuksia (ei toteutettu):

- Spring Cloud Config Server – keskitetty konfiguraationhallinta
- Circuit Breaker (Resilience4j) – vikaantumisen eristäminen
- Hajautettu lokitus ja tracing (Zipkin + Sleuth)
- Autentikointi ja auktorisointi (Spring Security / OAuth2)
- Kontainerisaatio (Docker / Kubernetes)

Yhteenveto

Tehtävässä toteutettiin toimiva Spring Cloud -mikropalveluarkkitehtuuri, jossa on Eureka-palvelurekisteri, kaksi forex-service-instanssia load balancerilla, currency-conversion-service Feign clientilla, Zuul API Gateway ja Image-service. Kaikki palvelut rekisteröityvät Eurekaan ja ovat saavutettavissa gatewayn kautta.